

PrismatAI Training Plan — Version 1 Draft

Date: 2026-03-06 **Status:** DRAFT — For review by human players, external AI reviewers, and domain experts

Approach: Track A — Supervised learning on human replays first; iterative self-play in a future phase **Author:** Surfinite + Claude Code

Important Note to Reviewers

This is the first training attempt using a fully accurate JavaScript game engine — a direct transpilation of the original ActionScript 3 source code. All previous training work (AWS binary shards, old feature schemas, prior model weights, prior win-rate statistics) has been discarded and is not referenced here. Please treat this as a clean start.

Key Verification Items

These are facts the plan depends on. If any are wrong, assumptions about data scale, feasibility, or success criteria may need revision. Reviewers should flag anything they cannot confirm or believe to be incorrect.

- **V1 — Usable human replay count:** ~104,675 rated games (both players 1500+ rating) were confirmed in the database as of March 5, 2026. A balance validation pass across ~58,297 additional post-patch codes is currently in progress. Expected usable pool upon completion: approximately **110,000 games**. This is the primary ceiling for supervised training data. VERIFY: confirm final count once validation completes.
- **V2 — Competitive unit count:** The authoritative unit list is `bin/asset/config/valid_units.json` and contains exactly **116 units**: 11 base set units (present in every game) and 105 dominion/random set units (the ranked competitive pool). This list was derived from the live game's unit selection screen, March 2026. The `cardLibrary.js` contains 161 entries but 45 of those are deprecated, event-mode, or removed units that no longer appear in competitive play. Any feature schema must be built on the **116-unit list**, not the raw `cardLibrary` count.
- **V3 — Balance validation definition:** A replay is considered balance-valid if: (a) it is post-patch (start time after January 14, 2019, ~18:00 UTC), OR (b) it is pre-patch and its deck does not contain any of the six units changed in that patch: Wild Drone, Odin, Militia, Mobile Animus, Sentinel, Blood Phage. VERIFY: confirm these are the only balance-relevant changes at that patch boundary and that the timestamp is correct.
- **V4 — JS engine accuracy:** The JavaScript engine is a transpilation of the original AS3 engine and is considered authoritative. It has been validated against replay data (100% pass rate on a sample of 500 replays). VERIFY: run validation on a fresh random sample of 1,000+ balance-validated codes before using engine output as training data, and confirm the pass rate is acceptably high.
- **V5 — MCDSAI as success benchmark:** The target for supervised training is: the trained model, used as the evaluation function in Alpha-Beta HPS search, wins a statistically significant majority of games against MCDSAI. VERIFY: What is MCDSAI's approximate playing strength relative to known baselines (e.g. OriginalHardestAI, or equivalent human rating)? A benchmark tournament (200+ games,

consistent think time) should be run before training begins so the success bar is anchored to a known reference point. If MCDSA I proves to be a poor benchmark (too weak or too strong), this criterion should be revised before committing to training.

- **V6 — Training record estimate:** Human replay extraction produces approximately one record per player-turn. At ~30 turns average game length, 110,000 games yields approximately **3.3 million training records** ($110,000 \times 30 \times 2 \text{ players} / 2$). For comparison, Churchill (2019) generated 15 million records from 500,000 AI self-play games. Our human dataset is therefore roughly 4–5x smaller. This is relevant to expected model quality and regularisation choices. VERIFY actual average game length from a sample of balance-validated replays.
- **V7 — PvP vs PvAI composition:** The 110K codes may include games played against MasterBot (the in-game AI) rather than human opponents. Human-vs-AI games have different strategic characteristics than PvP games. VERIFY: what fraction of the validated code pool is PvP vs PvAI? Decide whether to exclude PvAI games, use them with a separate label scheme, or treat them identically.
- **V8 — Rating floor:** Training uses games where both players have 1500+ rating (~110K games). Higher floors are: 1800+ (~72K games), 2000+ (~24K games). VERIFY: does the Prismata player community consider 1500-rated play strategically sound enough for training a strong evaluator, or would a higher floor produce better value targets at the cost of fewer records?
- **V9 — Supplementary data: human vs MasterBot replays:** A pool of games between human players and the in-game MasterBot AI may exist and could supplement training. These would provide AI-level value targets (MasterBot plays near the quality of the existing AI). VERIFY: are these available, and how many? Note as a potential data source if the human-only dataset proves insufficient.

Overview

The goal is to train a neural network evaluation function for Prismata that, when used inside the existing Alpha-Beta Hierarchical Portfolio Search (HPS) engine, produces an AI stronger than the MCDSA I baseline.

Churchill (2019) demonstrated this approach works — training on 500K AI self-play games produced an evaluator that beat both the resource heuristic and the playout evaluator. We are building on that result with three key differences:

1. **Accurate engine:** The JS engine correctly implements all game rules. Prior C++ approximations had known errors.
2. **Human replay data:** Human expert games carry stronger strategic signal than AI self-play (the AI being trained against will be stronger than prior AI self-play partners were).
3. **Architecture exploration:** Churchill's 2-layer, 512-neuron MLP was chosen without exhaustive search. We will run targeted ablations before committing to a full training run.

Self-play iteration (AlphaZero-style) is planned as a future phase. This plan covers supervised training only.

A Note on Code Cleanup

The `cardLibrary.js` contains 161 entries, of which 45 are deprecated, event-mode, or non-competitive units. These units never appear in balance-validated competitive replays and should not appear in any training

feature vector.

Recommendation: Before finalising the feature schema, prune `cardLibrary.json` to remove the 45 non-competitive entries. This is not strictly required (the `valid_units.json` whitelist already filters them out at runtime), but it reduces confusion, eliminates a source of latent bugs, and produces cleaner code going forward. This can be done as a standalone task independently of the training pipeline.

Phase 0: Prerequisites and Baseline Measurements

Goal: Complete all setup before any training work begins. Nothing in Phase 1+ should start until all items here are done and recorded.

0a. Complete Balance Validation

- Run post-patch validation to completion across the remaining ~58,297 codes
- Import results into `replays.db`
- Produce final canonical code list: `balance_validated_1500plus.json`
- Record final confirmed count. If count differs from ~110K by more than 10%, revisit V1 assumptions.

0b. MCDSAI Benchmark Tournament

- Run 200+ games: a search-based AI (using OriginalHardestAI configuration as a reference player) vs. MCDSAI
- Record win rate with Wilson confidence interval
- This establishes the concrete baseline the trained model must exceed
- **Decision gate:** If MCDSAI is already far weaker than OriginalHardestAI, it may not be a meaningful training target. Equally, if it is significantly stronger, the bar may be set too high for a first supervised pass. Adjust the success criterion based on this result before proceeding.

0c. JS Engine Validation on Representative Sample

- Run replay validation on a fresh random sample of 1,000 balance-validated codes
- Record pass rate
- **Decision gate:** If pass rate is below 95%, investigate failures before proceeding. Systematic failures indicate an engine correctness issue that would corrupt training data.

0d. Dataset Composition Audit

- For the final code list: measure PvP vs PvAI ratio (V7)
- Measure rating distribution (how many 1500–1799, 1800–1999, 2000+)
- Measure game length distribution (average turns per game for record count estimate V6)
- Measure card set distribution (base-only vs random-set games)
- Record all findings. They directly inform feature schema choices and training split decisions.

0e. Optional: cardLibrary Cleanup

- Identify and remove the 45 non-competitive unit entries from `cardLibrary.json`
- Update any code that references them (JS engine, C++ loader)
- This is independent of the training pipeline; can be parallelised with other Phase 0 work

Phase 1: Feature Schema Design

Goal: Define a canonical, versioned feature vector before any data extraction begins. This schema must be agreed upon and frozen before Phase 2 starts. Changing the schema after extraction requires full re-extraction.

1a. The Unit Representation Problem

A game of Prismata uses 11 base set units plus 8 randomly selected units from the 105-unit dominion pool — so at most 19 unit types are present in any given game. However, different games use different random units. For the feature vector to be universal (usable across all card sets), it must accommodate all 116 possible unit types, with zero-counts for types not present in the current game.

Three representation options are presented for reviewer input:

Option 1 — Type-Count (Churchill 2019 Baseline)

For each of the 116 unit types, record how many units of that type each player owns. Add resource counts and the current player indicator.

- **Feature vector size:** 116×2 (count per player) + 12 (resources: 6 types $\times$ 2 players) + 1 (player to move) = **~245 features**
- **Advantages:** Simple, fast inference, established precedent. Churchill achieved competitive results with this approach.
- **Disadvantages:** Loses all instance-level information: which units are under construction, which have been used this turn, which are currently blocking, which have taken partial damage.
- **Churchill's note:** "This encoding discards information such as which units may be activated, individual unit instance properties... since the states are all recorded at the beginning of each turn when units are not yet activated, much of the effect of this information loss is alleviated." However, our training data may include mid-turn states (one record per action, not only per turn), making this loss more significant.

Option 2 — Type-Count with Aggregate Instance Flags (Recommended Starting Point)

Extend Option 1 with per-type aggregate flags. For each unit type and each player, also encode: count under construction, count with ability used, count currently blocking, count with partial damage.

- **Feature vector size:** 116×2 players $\times$ 5 values (total, building, ability-used, blocking, damaged) + 12 resources + 1 player = **~1,173 features**
- **Advantages:** Captures tactically relevant state without per-instance complexity. Encodes information the model genuinely needs: e.g. how many Rhinos are tapped, how many Forcefields are still building, how many units took partial breach damage.
- **Disadvantages:** ~5 $\times$ larger than Option 1; more expensive to extract and slower to infer (though still well within C++ inference speed requirements at this scale).
- **Why this is the recommended starting point:** The JS engine reliably exposes all these per-instance fields. Using them is the primary advantage of the accurate JS engine over the prior C++ approximation. Option 1 would replicate Churchill but not use our engine's accuracy advantage.

Option 3 — Per-Instance Encoding with Set Aggregation

Encode each unit instance individually and aggregate using a permutation-invariant method (sum/mean pooling, or an attention mechanism as explored in the SAINT paper (2025)).

- **Advantages:** Maximum expressiveness; no information loss.
 - **Disadvantages:** More complex architecture and more difficult C++ deployment. Variable-length input requires careful handling.
 - **Recommendation:** Treat as a future experiment if Options 1 and 2 plateau. Not the starting point for iteration 1.
-

Reviewer question: Is Option 2 the right starting point, or is there a strong argument for starting with Churchill-exact (Option 1) or the per-instance approach (Option 3)?

1b. Label Design

- **Training target:** probability that the active player wins the game from this state
- **Raw label:** 1 if active player won, 0 if lost (draw = 0.5)
- **Output activation:** sigmoid (range 0–1) or tanh (range –1 to 1) — both work; Churchill used tanh. Standardise and document before training.
- **Loss function:** MSE on value prediction. Churchill found MSE and binary cross-entropy produce similar results for this task.

1c. Schema Versioning

- Schema stored as a single versioned JSON file: `training/schema_v1.json`
 - Must include: unit list (ordered), feature names and indices, value ranges, normalisation method, label encoding
 - Any change to the feature vector = new version. Old data cannot be mixed with new schema.
 - The canonical unit ordering comes from `valid_units.json` (base set first, then random set alphabetically, or by some consistent ordering). This ordering must be fixed and documented.
-

Phase 2: Data Extraction Pipeline

Goal: Extract all usable human replay data through the accurate JS engine into a single canonical dataset. All previous JSONL files are considered legacy and are not mixed into this output.

2a. Extraction

- **Input:** `balance_validated_1500plus.json` from Phase 0a
- **Process:** For each replay code, fetch from S3, step through game using JS engine, record one training record per player-turn
- **Output:** Single canonical JSONL: `training/data/human_replays_v1.jsonl` (chunked if needed for size)
- **Record format:** `{ "features": [...], "label": 0.0–1.0, "replay_code": "...", "turn": N, "rating_p0": X, "rating_p1": Y }`

- **Note:** Store raw state fields alongside the feature vector in each record if feasible. This allows re-vectorisation (re-running the featurisation step without re-fetching replays) if the schema changes during Phase 3 ablations.

2b. Train / Validation / Test Split

- Split by replay code (not by individual record) to prevent game-level data leakage
- Suggested split: 80% train / 10% validation / 10% test
- Test set held out entirely — not used during Phase 3 architecture search
- Consider a temporal split (earlier games train, more recent validate/test) as an alternative that better reflects deployment conditions

2c. Deduplication

- Some codes may appear across multiple source lists. Deduplicate by code before extraction begins.
- After extraction, verify no duplicate records (same replay_code + turn combination)

2d. Post-Extraction Validation

- Spot-check 100 random records against expected game outcomes
- Verify label distribution (~50/50 expected given symmetrically-rated games)
- Verify feature value ranges (no NaN, no Inf, no obviously wrong counts)
- Verify total record count is consistent with estimate from V6

Phase 3: Architecture Search and Ablation

Goal: Determine a good architecture before a full training run. Use a representative subset of training data. Do not proceed to Phase 4 without reviewing results here.

This phase is the primary place where the plan should be adjusted based on empirical findings.

3a. Churchill Baseline (Lower Bound)

Replicate Churchill (2019) as closely as possible on our data:

- Feature set: Option 1 (type-count, ~245 features)
- Architecture: 2 hidden layers, 512 neurons, tanh activation
- Optimiser: Adam, LR = 1e-5
- Loss: MSE, value prediction only

This is the lower bound. If a more complex model cannot beat it, something is wrong with the model, the data, or the problem formulation.

3b. Feature Richness Ablation

Train same Churchill architecture on Option 2 features (~1,173 features) and compare validation accuracy.

- If Option 2 adds less than ~1 percentage point: consider whether the complexity is justified
- If Option 2 adds ≥ 2 percentage points: Option 2 is likely the better starting point

3c. Capacity Search

Vary hidden dimension: 128, 256, 512, 1024. Vary depth: 2, 3, 4 layers. Use the winning feature set from 3b.

Key question: is 512 neurons (Churchill's choice) actually optimal, or is it over/under-powered for a ~245 or ~1,173-feature input? A smaller model infers faster in C++ (directly increases search depth at fixed think time). A 256-neuron model may be preferable to a 512-neuron one if accuracy is similar.

3d. Policy Head Experiment (Optional)

Optionally add a policy head predicting which move the human took. If move-prediction accuracy reaches ~30%+, this head may improve search quality via PUCT-style move ordering (AlphaZero approach). If accuracy is below ~20%, the signal is too weak to help. Low priority for iteration 1 but worth a quick test.

3e. Architecture Decision Gate

Before Phase 4, review:

- Which feature set won (Option 1 vs 2)?
 - What is the optimal depth and width?
 - Does validation accuracy plateau before the full dataset? (If yes: data-limited, not capacity-limited — consider self-play data generation sooner)
 - Is a policy head worth including?
-

Phase 4: Full Training Run

Goal: Train the chosen architecture from Phase 3 on the full training set.

4a. Training

- Full training set (80% of extracted records)
- Validate every epoch on validation set
- Save best checkpoint by validation accuracy
- Early stopping with patience ~10 epochs

4b. Regularisation

With ~3.3M records vs Churchill's 15M, overfitting risk is higher. Apply:

- Dropout: 0.1–0.3 on hidden layers
- Weight decay: L2 regularisation (1e-4 to 1e-5)
- Label smoothing: 0.05–0.15 (replaces hard 0/1 targets with softened values)

Specific values should be set based on Phase 3 observations.

4c. Learning Rate Schedule

Churchill used a fixed Adam LR of 1e-5. Cosine decay or step decay may improve convergence — worth a quick comparison during Phase 3 before locking in.

4d. Infrastructure

- Local: Intel Arc B580 (12GB VRAM, XPU backend)
- Cloud: AWS g4dn.xlarge or GCP g2-standard-4 if local is insufficient
- Monitor training vs validation curves throughout; any run where training accuracy significantly exceeds validation accuracy early should be investigated before completing

4e. Export and C++ Integration

- Export weights to binary format compatible with `NeuralNet.cpp`
- Verify all expected tensors are present (26 tensors required by C++ loader)
- Smoke test: load weights in C++, evaluate a known state, confirm output is in expected range

Phase 5: Evaluation

Goal: Measure trained model strength against MCDSAI and assess whether success criteria are met.

5a. Tournament Setup

- Players: `PrismatAlpha_AB` with trained neural net eval vs. MCDSAI
- Games: minimum 500 for a meaningful confidence interval (at 500 games, a 5pp win rate difference is detectable at 95% confidence)
- Card sets: random card sets matching MCDSAI's normal play conditions
- Think time: 3 seconds per turn (consistent with Phase 0b benchmark)
- Report Wilson confidence interval, not just point estimate

5b. Success Criteria

- **Pass:** Win rate vs MCDSAI above 50% with 95% confidence (lower bound of Wilson CI > 0.50)
- **Partial pass:** Above 50% but not yet statistically significant — run more games
- **Fail:** Below 50% — investigate data pipeline and Phase 3 findings before concluding model is fundamentally too weak

5c. Secondary Evaluations

- Run vs OriginalHardestAI as a stable reference for any future comparability
- Run model vs itself (expected ~50%, confirms basic sanity)
- If Prismata players are available, qualitative play sessions against the model are valuable for assessing strategic coherence that statistics may miss

5d. Decision Gate: Proceed or Iterate?

- **Success:** Proceed to Phase 6 (self-play iteration)
 - **Failure:** Diagnose root cause. Return to Phase 3 (architecture) if model architecture is suspected; Phase 1 (feature schema) if features seem inadequate; Phase 2 (data quality) if the pipeline seems suspect.
 - **Partial success:** Begin self-play data generation with current model and assess whether one iteration of self-play improves it before fully rearchitecting.
-

Phase 6: Self-Play Iteration (Future Phase — Outline Only)

Out of scope for this draft but documented so reviewers can assess whether the Track A plan sets it up correctly.

6a. Self-Play Data Generation

- Use trained model as evaluation function in the JS engine matchup runner
- Generate a target of at least 100,000 self-play games (roughly matching human dataset size) before retraining
- Store in same schema format as human replay training data for compatibility

6b. Iterative Training

- Retrain on a mix of human replay data + self-play data
- Maintain human replay data in the mix throughout (never below 20% human data, following AlphaZero practice)
- Evaluate each iteration against its predecessor and against MCDSA

6c. Gumbel AlphaZero Consideration

The Gumbel AlphaZero paper (Danihelka et al., ICLR 2022) proposes improved exploration during self-play by sampling actions without replacement, providing policy improvement guarantees even with few simulations. This is directly applicable to the HPS search framework and should be evaluated for self-play iteration 2+.

Open Questions for Reviewers

Input is explicitly solicited on the following. These do not have agreed answers.

1. **Feature richness:** Is Option 2 (type-count + aggregate instance flags) the right starting point, or should we begin with Churchill-exact (Option 1)? Is there a strong argument for starting with per-instance encoding (Option 3)?
2. **Rating floor:** Should training use 1500+ (~110K games) or 1800+ (~72K games)? Higher-quality data vs. more data — what does practical Prismata experience suggest about the strategic soundness of 1500-rated play?
3. **Card set filtering:** Should training be limited to games that use the standard random-set format? Base-only games (no random units, both players use only the 11 base set cards) are strategically very different. Should they be excluded, used separately, or included?
4. **Value target:** Churchill predicted game winner as the training target. An alternative is to predict final resource advantage (a graded signal rather than binary). Which is more appropriate for Prismata, where games are often strategically decided well before they technically end?
5. **Architecture prior:** Churchill used a plain MLP. Leela Chess Zero and AlphaZero use residual networks. Is there a theoretical or empirical reason to prefer ResNet-style skip connections for this problem, or is a plain MLP sufficient given the flat (non-sequential) nature of the feature vector?

6. **MCDSAI calibration:** Is beating MCDSAI a meaningful training milestone? What do experienced Prismata players estimate MCDSAI's skill level to be in terms of human rating equivalence?
-

What This Plan Does Not Cover

- Specific hyperparameter grid search methodology (to be detailed after architecture is selected in Phase 3)
 - Cloud compute provisioning specifics (to be planned once data scale is confirmed in Phase 0)
 - C++ NeuralNet integration changes (existing infrastructure reused; only weight file format must remain compatible)
 - Multi-GPU training (not needed at this data scale)
 - Policy learning beyond move ordering in search
-

References

- Churchill, D. & Buro, M. (2015). *Hierarchical Portfolio Search: Prismata's Robust AI Architecture for Games with Large Search Spaces*. AIIDE 2015.
- Churchill, D. (2019). *Machine Learning State Evaluation in Prismata*. AIIDE Workshop 2019.
- Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel*. ICLR 2022.
- Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero)*. ICML 2021.
- Landers, M. et al. (2025). *SAINT: Attention-Based Policies for Discrete Combinatorial Action Spaces*.
- Sokota, S. et al. (2025). *Superhuman AI for Stratego Using Self-Play Reinforcement Learning and Test-Time Search*.